E08 — Network Delay Time (Dijkstra's Algorithm)

Experiment: 8 | **LeetCode:** #743 | **CO:** CO3, CO4 | **BT Level:** BT5

Problem Statement

You are given a network of `n` nodes, labeled from `1` to `n`. You are also given `times`, a list of travel times as directed edges `times[i] = (ui, vi, wi)` where `ui` is the source node, `vi` is the target node, and `wi` is the travel time.

We will send a signal from node `k`. Return the **minimum time it takes for all n nodes to receive the signal**. If it is impossible for all nodes to receive the signal, return `-1`.

Example 1:

```
Input:  times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
Output: 2
```

Example 2:

```
Input:  times = [[1,2,1]], n = 2, k = 2
Output: -1
```

Example 3:

```
Input:  times = [[1,2,1]], n = 1, k = 1
Output: 0
```

Concept Review: Dijkstra's Algorithm

Find shortest paths from source `k` to all nodes. The answer is the **maximum** of all shortest distances (if any node is unreachable → return -1).

Algorithm (min-heap):

```
dist[k] = 0; dist[others] = ∞
Push (0, k) onto min-heap

while heap not empty:
    d, u = pop minimum
    if d > dist[u]: skip (stale entry)
    for each neighbor v with edge weight w:
        if dist[u] + w < dist[v]:
            dist[v] = dist[u] + w
            push (dist[v], v) onto heap

answer = max(dist[1..n]) if no ∞ else -1
```

Implementation

```
import heapq
from collections import defaultdict

def networkDelayTime(times, n, k):
    """
    Dijkstra's algorithm for single-source shortest paths.
    Time: O((V + E) log V)   |   Space: O(V + E)
    """
    # Build adjacency list
    graph = defaultdict(list)
    for u, v, w in times:
        graph[u].append((v, w))

    # Initialize distances
    INF = float('inf')
    dist = {node: INF for node in range(1, n + 1)}
    dist[k] = 0

    # Min-heap: (distance, node)
    heap = [(0, k)]

    while heap:
        d, u = heapq.heappop(heap)

        # Skip stale entries (we already found a shorter path)
        if d > dist[u]:
            continue

        # Relax edges from u
        for v, w in graph[u]:
            new_dist = dist[u] + w
            if new_dist < dist[v]:
                dist[v] = new_dist
                heapq.heappush(heap, (new_dist, v))

    max_dist = max(dist.values())
    return max_dist if max_dist < INF else -1
```

Detailed Trace

Input: `times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2`

Graph (adjacency list):

```
2 → [(1, 1), (3, 1)]
3 → [(4, 1)]
```

Initial state:

```
dist = {1: ∞, 2: 0, 3: ∞, 4: ∞}
heap = [(0, 2)]
```

Step	Pop	d	u	Relax edges	heap (after)	dist
1	(0,2)	0	2	2→1: 0+1=1 < ∞; 2→3: 0+1=1 < ∞	[(1,1),(1,3)]	{1:1,2:0,3:1,4:∞}
2	(1,1)	1	1	no outgoing edges from 1	[(1,3)]	unchanged
3	(1,3)	1	3	3→4: 1+1=2 < ∞	[(2,4)]	{1:1,2:0,3:1,4:2}
4	(2,4)	2	4	no outgoing edges from 4	[]	unchanged

Final distances: `{1:1, 2:0, 3:1, 4:2}`

```
max_dist = max(1, 0, 1, 2) = 2
```

Output: 2 ✓

Trace Example 2: `times = [[1,2,1]], n = 2, k = 2`

```
graph: 1 → [(2, 1)]  (no edge from 2)
dist = {1: ∞, 2: 0}
heap = [(0, 2)]

Pop (0, 2): no outgoing edges from 2
Heap empty.

dist = {1: ∞, 2: 0}
max_dist = ∞ → return -1 ✓
```

Path Reconstruction

```
def networkDelayTime_with_path(times, n, k):
    """Extended version that also returns the path to furthest node."""
    graph = defaultdict(list)
    for u, v, w in times:
        graph[u].append((v, w))

    INF = float('inf')
    dist = {node: INF for node in range(1, n + 1)}
    prev = {node: None for node in range(1, n + 1)}
    dist[k] = 0

    heap = [(0, k)]

    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
```

```
                heapq.heappush(heap, (dist[v], v))

    max_dist = max(dist.values())
    if max_dist == INF:
        return -1, []

    # Reconstruct path to furthest node
    furthest = max(dist, key=dist.get)
    path = []
    node = furthest
    while node is not None:
        path.append(node)
        node = prev[node]
    path.reverse()

    return max_dist, path
```

Test Suite

```
def test_network_delay():
    test_cases = [
        ([[2,1,1],[2,3,1],[3,4,1]], 4, 2, 2),
        ([[1,2,1]], 2, 2, -1),
        ([[1,2,1]], 1, 1, 0),
        ([[1,2,1],[2,3,2],[1,3,4]], 3, 1, 3),   # path 1→2→3 = 3 < 1→3
        ([[1,2,1],[2,1,3]], 2, 2, 4),             # 2→1→2... but Dijkst
    ]

    for times, n, k, expected in test_cases:
        result = networkDelayTime(times, n, k)
        status = "PASS" if result == expected else "FAIL"
        print(f"{status}: networkDelay(n={n}, k={k}) = {result} (expect

test_network_delay()
```

Complexity Analysis

Metric	Value
Time	O((V + E) log V) — each edge relaxation may push to heap
Space	O(V + E) — adjacency list + heap + dist array

Key Takeaways

- **Network delay = maximum shortest distance** from source to any node.
- Dijkstra's greedy approach works because edge weights are non-negative.
- **Stale entry check** (`if d > dist[u]: continue`) is critical for correctness and efficiency.
- If any node remains at ∞, it's unreachable → return -1.

- Use `defaultdict(list)` for the graph to avoid key existence checks.

References

- LeetCode #743 — Network Delay Time
- L35.md (Dijkstra's Algorithm) — full algorithm theory and proofs
- Cormen et al., *Introduction to Algorithms*, Ch. 24.3